Design Patterns

Design Patterns : Definition

- Documenting solutions to “common” problems
- Formalized best practices
- Usually documented with UML
- Usually classified as:
  - Creational Patterns : To create objects effectively
  - Structural Patterns : To organize classes and objects
  - Behavioral Patterns : Common communication patterns between objects
  - Concurrency Patterns : how to properly manage concurrency
- Seminal book on the topic : *Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN 978-0-201-63361-0.*

Singleton

- Creational pattern
- To guarantee that a given class only has one instance
- Warning: singleton is difficult to get right

CREATED WITH YUML

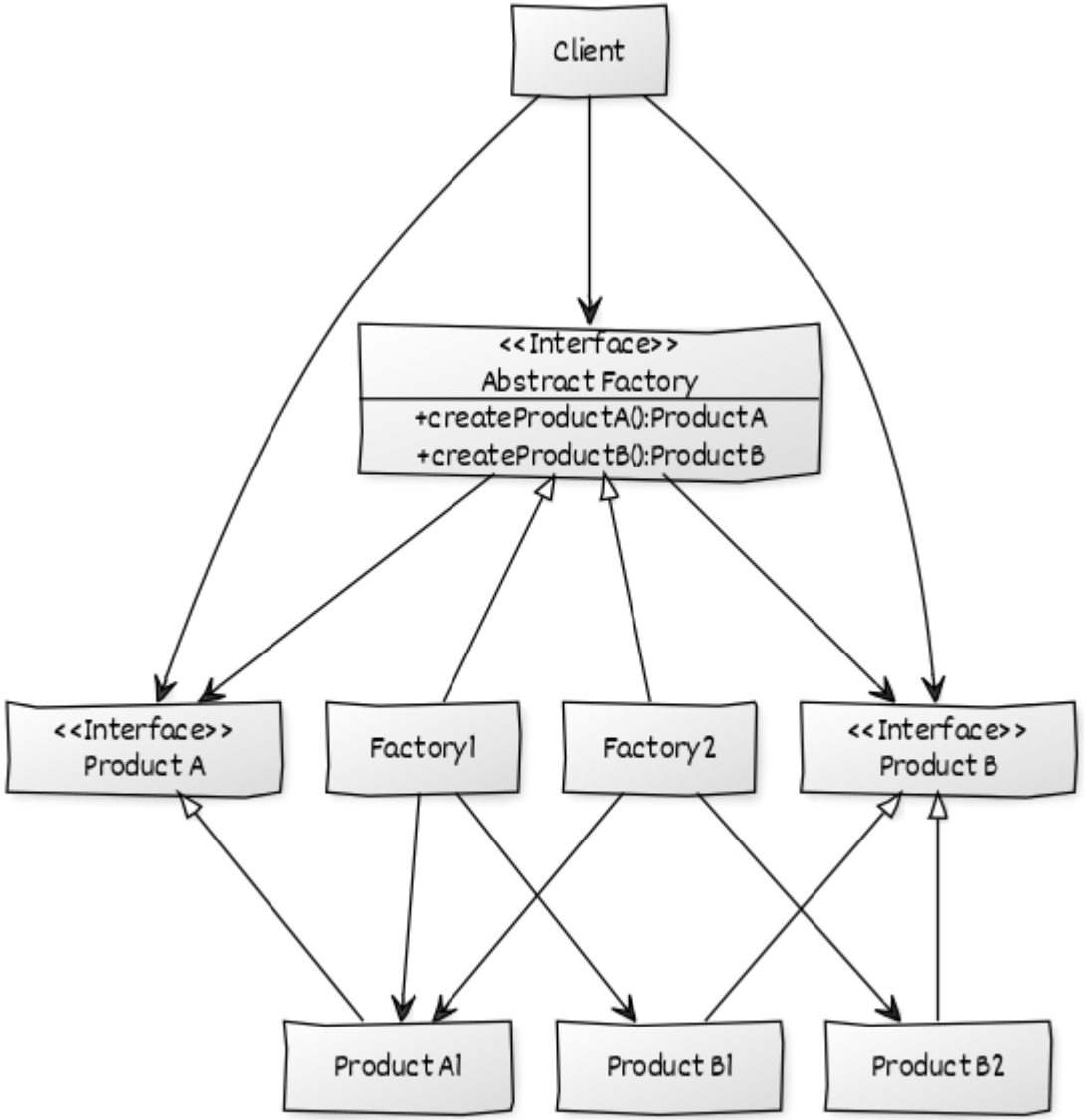
Singleton

Let’s code it together

Abstract Factory

- Creational pattern
- To avoid having the client to know the implementation
- To create “families” of object
  - Allows to “easily” change the implementation w/o having to update hundreds of lines of code
  - Allow to “easily” replace real implementation by mocks
- Base for many other patterns
  - Inversion of control a.k.a. dependency injection

Abstract Factory



Abstract Factory

```
public interface Animal { String getAnimal(); String makeSound(); }

public class Duck implements Animal {
   public String getAnimal() {return "Duck
   public String makeSound() {return "Squeks";}
}

public class Canard implements Animal {
   public String getAnimal() {return “Canard
   public String makeSound() {return “Coin Coin";}
}

public interface AbstractFactory {
   Animal create() ;
}
```

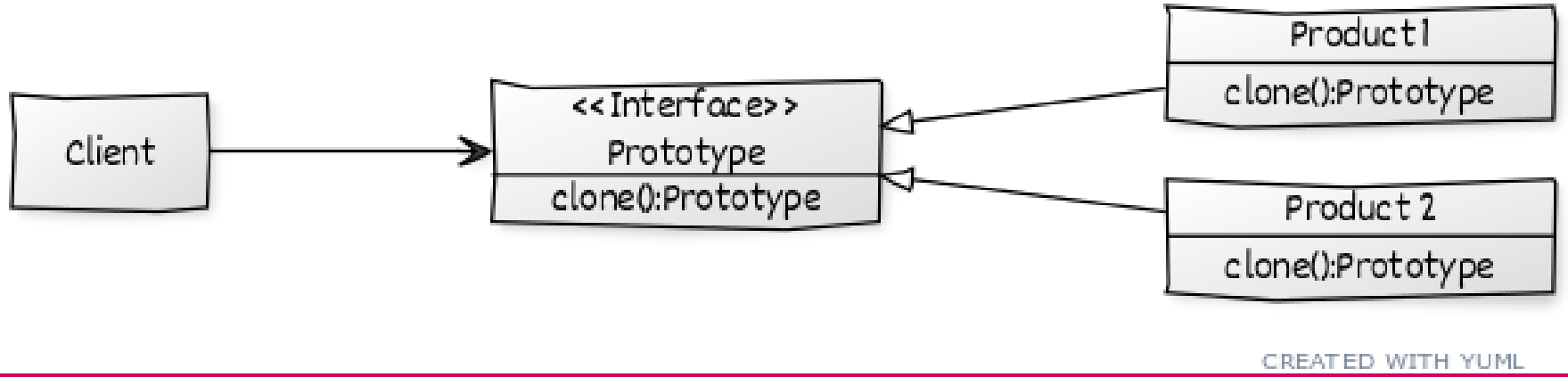
```
public class ENAnimalFactory implements AbstractFactory {
   public Animal create() { return new Duck();  }
}

public class FRAnimalFactory implements AbstractFactory {
   public Animal create() { return new Canard();  }
}

public class FactoryProvider {
   public static AbstractFactory getFactory(String choice){
      if(“EN".equalsIgnoreCase(choice)){
         return new EnAnimalFactory();
      } else if(“FR".equalsIgnoreCase(choice)){
         return new FRAnimalFactory();
      }
   }
}
```

Prototype

- Creational pattern
- Used to avoid the cost of initialization of complex structures by copying and modifying instead of creating from scratch
- Very often used for complex graph structure



Prototype

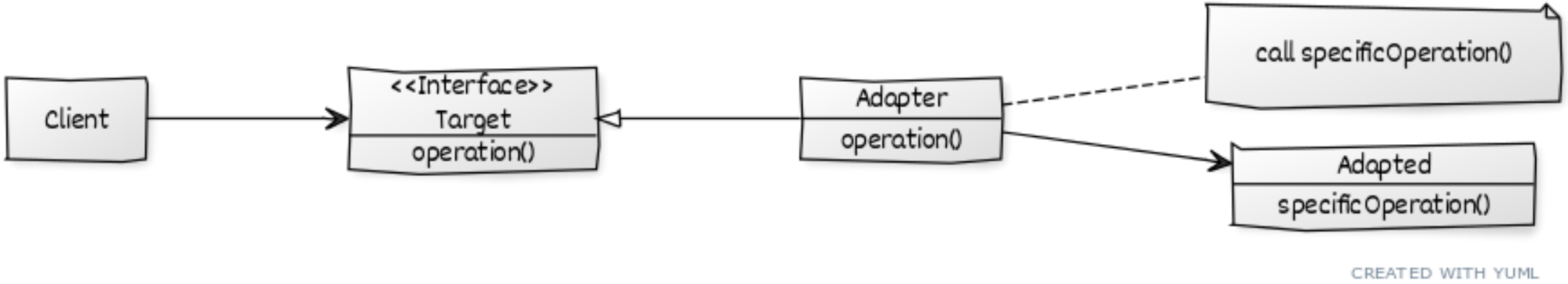
```
public abstract class Prototype implements Cloneable {
   public Prototype clone() throws CloneNotSupportedException{
      return (Prototype) super.clone();
   }
}

public class ConcretePrototype1 extends Prototype {
   public Prototype clone() throws CloneNotSupportedException {
      return (ConcretePrototype1) super.clone();
   }
}

public class ConcretePrototype2 extends Prototype {
  public Prototype clone() throws CloneNotSupportedException {
    return (ConcretePrototype2) super.clone();
  }
}
```

Adapter (or Wrapper)

- Structural pattern
- Convert one contract into another so that the converted one can be used in existing algorithms.

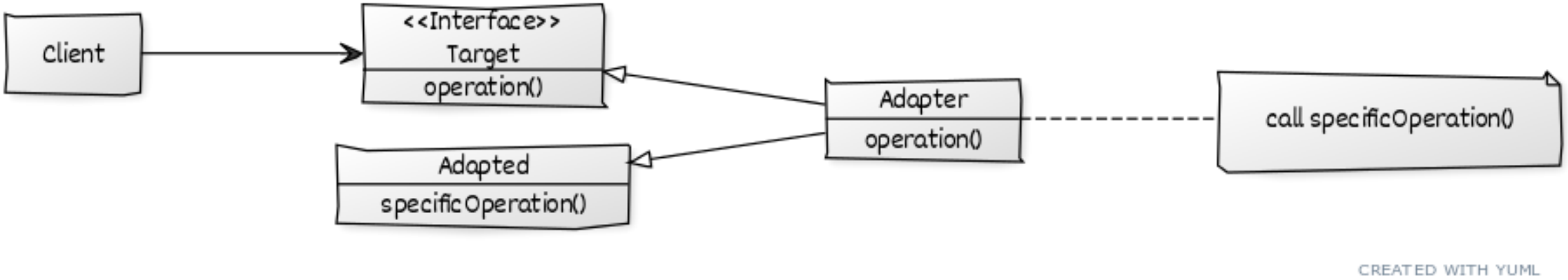


Adapter (or Wrapper)

```
interface Shape {
   public double computeArea()
}

public class Rectangle implements Shape {
   public double computeArea() {…}
}
public class Triangle implements Shape {
   public double computeArea() {…}
}
public class Circle {
   public double computeSurface() {…}
}

public class CircleAdapter implements Shape {
   public double computeArea() { return computeSurface() }
}
```

```
public class Client {
   public double computeAreaOfShapes() {
           List<Shape> shapes = new ArrayList<>();
           shapes.add(new Rectangle());
           shapes.add(new CircleAdapter());
           shapes.add(new Triangle());
           return shapes.stream().mapToDouble(
                     f -> f. computeArea()).sum();
   }
}
```

Adapter (or Wrapper) alternative